ERROR-BOUNDED SCIENTIFIC COMPRESSION

tang-compress

In-band prediction, learned integer context models, and a bit-exact GPU decoder

VS SPERR, 80 DB	PSNR POINTS WON	MAX-ABS POINTS WON	CPU = GPU BYTES	LEARNED GPU DECODE
0.670 × bits	54/55 held-out	34/42 held-out	570/570 cases	1.10 GB/s

FROZEN CODEC · 14 HELD-OUT FIELDS · KERNEL THROUGHPUT ON NYX 512³ · RATES EXACT, TIMINGS ON A LOADED MACHINE

Abstract

tang-compress is an error-bounded lossy compressor for floating-point scientific fields, built on a Rust numerical stack. A CDF 9/7 wavelet feeds per-band integer linear prediction, a deadzone quantizer with transmitted centroids, a context-modelled binary range coder (hand-designed or a small learned integer MLP), and a max-abs correction stream; an encoder-side chooser picks between this and a Tucker codec. Every byte is counted, error is measured on decoder output, and all probability-to-bit arithmetic is integer.

We tuned on five SDRBench fields, froze the codec, and evaluated on 14 held-out fields, 11 of them from four applications never seen in development. The frozen chooser uses 0.67–0.72× SPERR's bits at matched PSNR (60–100 dB) and 0.74–0.81× the per-point best external codec, winning 54 of 55 PSNR points and 34 of 42 max-abs points. A frozen pretrained learned model transfers: 0.83–0.89× SPERR, against 0.87–0.97× for the hand model.

A GPU variant's integer decoder is byte-identical between CPU and GPU on every tested case. It decodes at 1.3–7.3 GB/s (kernel) with static priors and 1.10 GB/s with a learned model compiled into the shader. The GPU formats cost 11–15% in rate against our serial coder, roughly SPERR parity. The CPU encoder is about 40× slower than SPERR.

PROVENANCE

Every number is copied from four reports in `docs/compression-math-survey/` or the CSV/JSONL files they cite. The only new runs are the visual examples (§5.4), logged in demo_results.json.

M0 · tang-compress-qtt-m0.md
M1 · tang-compress-m1.md
S · tang-compress-m1-summary.md
M1d · tang-compress-m1d.md
B · tang-compress/bench/out/figures · make_figures.py

1

Introduction

Simulations and instruments now produce data faster than it can be stored, moved or read back. A single cosmology or climate run writes more floating-point values than its machine can archive. Lossless coders help a little; on noisy floating-point mantissas they rarely get much past two to one. The field has therefore settled on **error-bounded** lossy compression. The scientist states how wrong a reconstructed value is allowed to be, either pointwise or as a root-mean-square target, and the compressor spends as few bits as it can while honouring that promise.

Four families of compressor define current practice, and each makes a different bet about where the redundancy in scientific data lives. Prediction-based coders such as SZ and its modular successor SZ3 [Di16; Liang22] bet on local smoothness: they predict each value from neighbours that have already been decoded, quantize the residual against the bound, and entropy-code what is left. Block-transform coders such as ZFP [Lindstrom14] bet on speed and random access: a fixed decorrelating transform on 4^d blocks, coded bit plane by bit plane, decodes fast from any position but pays for its simplicity in rate. Wavelet coders such as SPERR [Li23] bet on multiscale structure: a CDF 9/7 transform concentrates energy into a few coefficients, SPECK-style set partitioning [Pearlman04] codes where they are, and an outlier stream enforces a pointwise bound after the fact. Tensor coders such as TTHRESH [BallesterRipoll20] bet on low rank: an HOSVD core coded by bit planes is very strong on fields that factor well along their axes, but offers no pointwise mode.

Our starting point is the third family, and a narrow question about it. Wavelet coders spend most of their engineering on the transform and on set partitioning, and comparatively little on the final step that turns quantized coefficients into bits. SPECK codes significance and magnitude, but it does not exploit the fact that neighbouring coefficients inside one band are strongly correlated with each other [M1 §1]. **How much rate is left in that last stage, and can a coder that recovers it remain deterministic enough to be decoded bit-for-bit on a GPU?** The second half of the question matters because a compressor is only useful if its output can be read back quickly, and the fastest hardware available to most scientists is a GPU whose floating-point behaviour differs from the CPU that encoded the data.

This paper answers both halves with measurements. The answer to the first is that a good deal of rate is left. Per-band integer prediction plus a richer context model, hand-designed or learned, lets the wavelet coder use about a third fewer bits than SPERR at matched quality on fields it was never tuned on. The answer to the second is yes, at a cost. An integer-only decoder is byte-identical across CPU and GPU in every case we tested and decodes at more than a gigabyte per second, but its parallel stream format gives back roughly half of the rate advantage.

We make four contributions. First, a compressor in which every stage, from transform to range coder, is fully bit-accounted, with a max-abs mode and an encoder-side chooser between wavelet and Tucker codecs (§3). Second, an evaluation protocol in which the codec is frozen after tuning on five fields and then measured on fourteen it has never seen, eleven of them from applications absent from development (§4–5). Third, two GPU stream formats whose integer decoders are byte-identical on CPU and GPU, including a learned context model compiled into the shader (§3, §5). Fourth, an account of where the method loses, which we think is as useful as where it wins (§6).

2

Notation and measurement

We use one set of definitions for every codec, and measure every codec on its decoded output rather than trusting its own report. **Rate** is bits per value (bpv): the full size of the compressed file in bits, divided by the number of values, so headers, side information and model weights are all included. **PSNR** is $20 \log_{10}(\text{range} \,/\, \text{RMSE})$, where range is max − min of the original field. A **max-abs** bound is $b = \text{rel} \cdot \text{range}$ with rel ∈ {1e-2, 1e-3, 1e-4}, and a run counts only if every reconstructed cell lies within b of the original. [M0 §2]

Codecs expose different knobs, so we never compare them knob for knob. Each codec is swept over its own settings, its measured (PSNR, bpv) points form a Pareto front, and the rate at a target PSNR is a log-linear interpolation on that front. When we summarise across fields we use the geometric mean of bpv ratios, codec over reference, over the fields where both are defined. A ratio of 0.7 means 30% fewer bits. [M1; S]

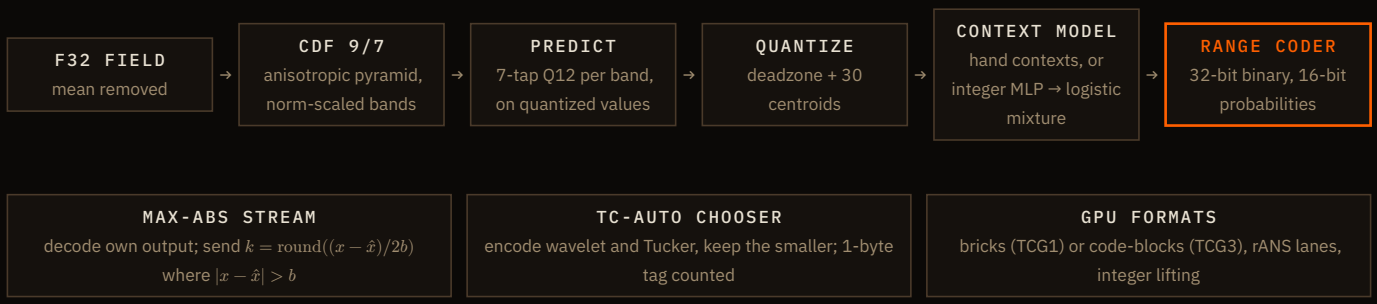


FIG 1 The pipeline. Every byte from every stage passes through one bit-accounting writer, so the reported rate has no side channel. [M1 §1; M1d §1]

3

Method

The compressor is a pipeline of five stages (Figure 1). A field is transformed by a wavelet, each band is predicted from itself, the residuals are quantized, a context model assigns a probability to every binary decision, and a range coder turns those probabilities into bits. Two further pieces sit beside the main line: a correction stream that enforces pointwise bounds, and a chooser that tries a Tucker codec as well and keeps whichever stream is smaller. Every byte from every stage passes through one bit-accounting writer, so there is no side channel through which rate could escape the count. [M1 §1]

3.1 Transform

We use the CDF 9/7 biorthogonal wavelet [Cohen92], implemented by lifting [Sweldens96] in IEEE double precision with whole-sample symmetric extension. Lifting lets the transform handle any length, odd ones included, without padding the field. An integer LeGall 5/3 transform gives a lossless mode through the same coder. [M1 §1]

Scientific fields are rarely cubes, so the pyramid is allowed to become anisotropic. At each level, every axis that is still at least eight samples long is split, and shorter axes stop. A 64×128×128 crop therefore gets four levels along its short axis and five along the others. Coefficients are scaled by the norm of their band's synthesis basis, so a single quantizer step produces the same mean-squared error in every band, and the mean of the field is removed before the transform. [M1 §1]

3.2 Per-band integer prediction

The idea that matters most in this paper is also the simplest. After a wavelet transform, a highpass band of a smooth field is not noise. Along the axes on which it was lowpass filtered it is itself smooth, with neighbour correlations around 0.9, and along the axis on which it was highpass filtered neighbours are anti-correlated, at around −0.5 [M1 §1]. A coder that codes each coefficient's magnitude from scratch throws that structure away.

We therefore fit a small causal predictor to each band. It has seven taps (the west, north and up neighbours, the three diagonals that share an edge with them, and the parent coefficient one level coarser), with least-squares weights stored as Q12 integers. Two details make it work. The weights are fitted on the **quantized** values that the decoder will actually see, not the true coefficients, so encoder and decoder predict identically. And a band keeps its predictor only if a cost proxy drops by more than 2%, so bands where prediction does not help pay nothing for it. The coder then codes the residual $q - \mathrm{pred}$, and all contexts are computed on residuals. [M1 §1]

On Hurricane-U at 80 dB this step alone moved the rate from 1.94 to 1.69 bpv, a 13% saving and the largest single gain we found during development [M1 §1]. We return to this point in the discussion: most of what follows, including the learned model, is refinement on top of it.

3.3 Quantization and rate control

Coefficients are quantized with a deadzone, $q = \mathrm{sgn}(c)\lfloor |c| \ / \Delta + 0.45 \rfloor$, which zeroes slightly more than a plain rounding quantizer would. On reconstruction, a value is not placed at the centre of its quantization bin but at the bin's measured centroid, sent in the stream for each combination of band class and |q| ∈ {1, 2, ≥3}: thirty offsets at 1/512 resolution. For PSNR or RMSE targets the step Δ is found by bisection on the RMSE of the actual decoder output, so the target is met by the real decoder, not by an estimate. [M1 §1]

3.4 Context modelling and coding

The coder is an integer, carry-propagating 32-bit binary range coder (the construction used by LZMA) with 16-bit

probabilities and two-rate adaptive bit models. Every residual is broken into binary decisions: whether it is zero, a short unary code on $(|r|-1) \gg k$, a few nearly raw low bits, and a sign. The Golomb parameter k adapts to the magnitudes of the causal neighbours, the parent and the prediction. [M1 §1]

What makes a context coder good is what it conditions on. Ours conditions on the subband class (three level groups by three orientation counts, plus the LL band), on the activity of eight causal neighbours in the same band, on the parent and its six face neighbours one level up, and on the signs of the west, north and up neighbours. [M1 §1]

TABLE 1 Learned model modes. Overfit weights are shipped in the stream and counted in the rate. [M1 §3]

Mode	Shape	Params	Cost in stream
pretrained	37→64→32→5	4,677	8 B fingerprint
overfit	37→16→5	693	≈750 B, 0.0057 bpv

3.5 A learned integer context model

A hand-designed context model can only condition on what its designer thought of. As an alternative we train a small neural network to predict the distribution of each residual from 37 integer features: log-magnitudes of seventeen causal or coarser coefficients, the band's prediction, activity summaries, and a one-hot band class. The network is an MLP with 8-bit fixed-point weights and 64-bit integer accumulators, and its output is a two-component discretized logistic mixture over the residual, the output family introduced by PixelCNN++ [Salimans17]. [M1 §3]

The difficulty is not training the network but making its output reproducible. Every binary probability the coder uses is an integer ratio of cumulative masses read from Q24 and Q16 tables, and those tables are built using only the IEEE operations + − × / √, which are correctly rounded everywhere, so no maths library is involved. An optional 33-bucket integer adaptive probability map refines the network's output. Training minimises the cross-entropy of the real quantized residuals, measured in bits, and is deterministic for a given seed. [M1 §3]

The model comes in two modes (Table 1). A **pretrained** network is trained once on other fields and identified in the stream by an eight-byte fingerprint. An **overfit** network is trained on the field being compressed, at encode time, and its weights are entropy-coded into the stream and paid for in the rate, about 750 bytes. [M1 §3]

3.6 Pointwise bounds

A PSNR target constrains the average error; many users need a guarantee on every value. To provide one, we code the wavelet layer at a step proportional to the bound, Δ = f·b, for eight values of f between 1 and 5, and decode the result. Every cell whose error still exceeds b receives an integer correction $k = \mathrm{round}((x-\hat{x})/2b)$, which is enough to guarantee $|x-\hat{x}-2bk| \le b$. Corrections are coded as an adaptive significance map with eight neighbour contexts, plus their magnitudes, and the rare cells that rounding to f32 still pushes past b are stored raw. The encoder keeps the value of f that gives the smallest total stream and re-verifies the bound on the decoder output. This is the same idea as SPERR's outlier stream [Li23], with a context-coded significance map. [M1 §1]

3.7 Tucker, and choosing between codecs

Some fields are better described as low-rank tensors than as multiscale images. For those we include a Tucker codec in the style of TTHRESH [BallesterRipoll20; DeLathauwer00]. It computes an HOSVD from the mode Gram matrices, truncates ranks by rate–distortion, quantizes factors at equal slope with per-column DPCM, and then refits the core against the **quantized** factors before coding it with the wavelet coefficient coder. Tucker has no max-abs mode. [M1 §1]

Neither codec dominates, so the shipped configuration, **tc-auto**, simply encodes with both and keeps the smaller stream, paying one byte to say which it chose. There is no clever pre-screen: it costs a second encode, and in exchange it is never far from the better of its two candidates. [M1 §8]

3.8 Determinism

Every step from probability to bit is integer arithmetic. The CPU wavelet decoder uses plain double-precision lifting with no fused multiply-adds, so it produces the same output on any CPU. The one exception is Tucker's reconstruction, whose matrix multiply is bit-identical on a given CPU but may differ in the last place across instruction sets. Its bitstream is identical everywhere. [M1 §1]

3.9 Two GPU stream formats

The serial coder above decodes one coefficient after another, because every context depends on the coefficients before it. A GPU needs thousands of independent pieces of

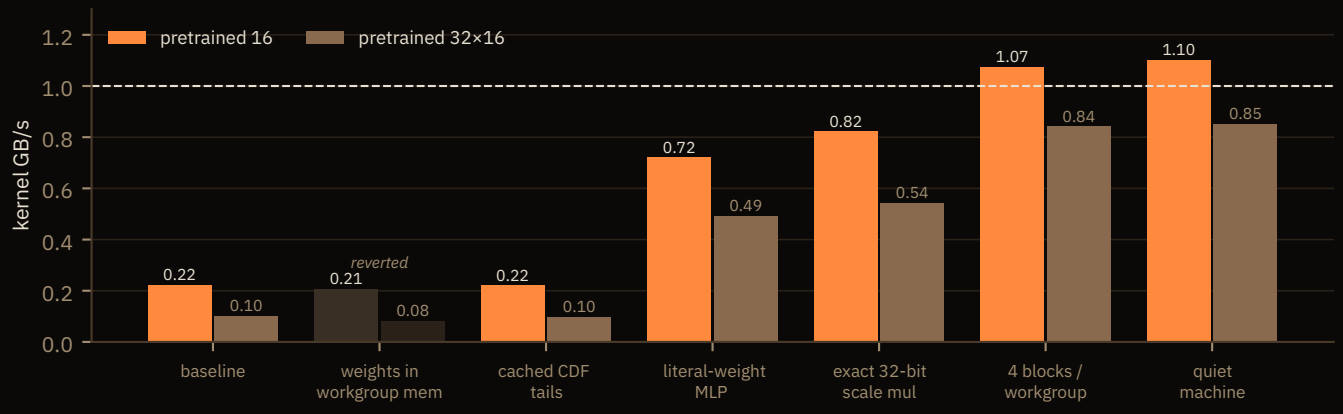


FIG 2 Learned TCG3 GPU decoder, kernel GB/s on NYX 512³ at 80 dB after each step. Dark bars were reverted: putting weights in workgroup memory reduced occupancy. Literal weights gave the largest single jump; packing four blocks per workgroup took the 16-hidden network past 1 GB/s. [M1d §11]

work. We built two stream formats that provide them, at some cost in rate.

The first, **TCG1**, splits the field into independent bricks, each with its own wavelet pyramid, which also gives random access. Inside a band, lines of coefficients are divided into hierarchical parity passes (seven in 3D) and dealt round-robin to 32 interleaved rANS lanes [Duda13], so that lanes can decode in parallel while still seeing coefficients from earlier passes. Our first version kept adaptive probability models per lane, and the GPU decoder spent its time moving model state through memory; it was about thirty times slower. Replacing adaptive models with static priors sent in the header cost only 1–4% in rate and made the lanes stateless. [M1d §1]

The decode itself is integer-only. Values live on a fixed-point grid $x \approx v \cdot 2^{e}$; dequantization uses a 30-bit mantissa per band; the inverse wavelet uses Q24 lifting constants through a 32×32→64-bit multiply, emulated with 16-bit limbs where the shading language lacks one. The final conversion to f32 is exact, because $|v| < 2^{24}$ and the scale is a normal power of two. Because the encoder rate-controls on this integer decoder, any loss from fixed point is already included in every rate we report. [M1d §1]

The second format, **TCG3**, removes the bricks. It uses one global fixed-point pyramid and tiles each subband into code-blocks of 16³ or 32³, in the spirit of JPEG 2000 [Taubman00]. A block may read any coarser coefficient but only its own region at its own level, so all blocks of a level are independent. TCG3 also adds residual-based contexts and sends a prior only when doing so saves bits. [M1d §9]

Finally we moved the learned model onto the GPU. The MLP runs entirely in 32-bit integers, under clamps on weights, activations and biases chosen so that no partial sum can overflow. Making it fast took three changes, shown step by step in Figure 2: compiling each stream's weights into the shader as literals, so the network becomes straight-line code with no weight loads; replacing an emulated 64-bit multiply with an exact 32-bit form; and packing four code-blocks into each workgroup so the GPU's 32-wide SIMD groups are full. Bitstreams were frozen before any of these changes and re-hashed after each one; every stream decoded identically throughout. [M1d §10–11]

4

Experimental protocol

The central risk in a paper like this one is that the codec has been tuned, consciously or not, to the fields it is evaluated on. We addressed that with a freeze.

All development used five fields: Miranda density and x-velocity, Hurricane Isabel U and TC, and NYX temperature, cropped to interior 64×128×128 windows from SDRBench's mirrors, plus four synthetic one-dimensional signals [M0 §2]. When development was finished we froze the codec at commit `4e27445` and froze a pretrained model, trained only on those five fields. Only then did we fetch fourteen further fields, listed in Table 2. Eleven come from four applications that played no part in development: SCALE-LETKF weather, CESM atmosphere, EXAFEL detector images and QMCPACK orbitals. Three come from the same simulations as the development fields and act as controls. One encoder-only crash fix landed during the held-out run; the development-field Tucker streams were re-verified to be byte-identical after it. [M1 §8]

TABLE 2 Held-out fields, fetched only after the freeze. [M1 §8]

Application	Held-out fields	Crop
NEW APPLICATIONS · 11		
SCALE-LETKF	T, U, QV, PRES	64×128×128
CESM-ATM	CLDTOT, FLDS, LHFLX, TS	1024² 2D
EXAFEL	event 0	1024² 2D
QMCPACK	orb 0, orb 100	115×69×69
SAME APPLICATIONS · 3		
Miranda	pressure	64×128×128
Hurricane	V	64×128×128
NYX	velocity_x	64×128×128

We compare against SPERR, TTHRESH, SZ3, ZFP and our own earlier QTT codec, each built from source or used through its library. SPERR was run in both its PSNR and its pointwise-error modes. TTHRESH has only a PSNR mode and accepts only inputs of three or more dimensions, so it has no results on the five 2D fields. SZ3 was run in PSNR and absolute-bound modes. For pointwise bounds, ZFP was given its best honest configuration: the largest tolerance whose **actual** maximum error stays within b. [M0 §2, §8]

The tools' own error reports were ignored. Every compressed file was decoded by its codec's decoder and measured by one harness, and every max-abs result was checked cell by cell. Rates come from file sizes, so our model fingerprints, shipped weights and chooser byte are all paid for, just as the baselines' headers are. [M0 §8.2; M1]

One caveat applies to every timing in the paper. The machine was shared with other work and heavily loaded throughout, with load averages of 20–35 during the CPU experiments and up to 70 during the GPU ones; repeated timings varied by up to a factor of two. Rates are exact. Timings should be read as orders of magnitude. [M1; M1d]

5

Results

5.1 A detour that set the direction

Before building the wavelet coder we built a bit-accounted quantized tensor-train (QTT) codec [Oseledets11], because tensor trains promise logarithmic storage for fields with the right structure. The result was clear (Figure 3). On real 3D fields at 60 dB and above, the frontier belongs to SPERR or TTHRESH everywhere: QTT needed two to three times their bits on Miranda and Hurricane, and six times or more on NYX. QTT does win on one-dimensional oscillatory and algebraic signals, by two to six times on damped sinusoids, but that is a niche, and the comparison is partly unfair to the other tools, which see those signals reshaped into 2D or 3D. We took this as evidence that a strong transform plus a strong entropy coder was the better bet, which is what the rest of this paper pursues. [M0 §8.3–8.5]

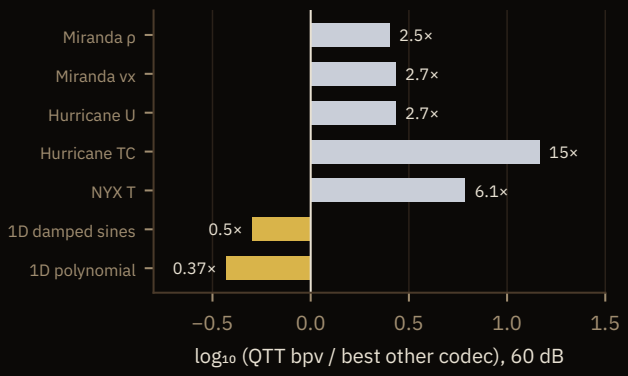


FIG 3 QTT bpv relative to the best other codec at 60 dB. Bars right of zero are losses. [M0 §8.3]

5.2 The development fields

Table 3 shows the classical stack, before any learning, against the external codecs on the development fields. On these fields tc-wavelet is already ahead of SPERR at every point but one (Hurricane U at 60 dB, +4.7%), and Tucker is ahead of TTHRESH at 100 dB on Miranda by 11% (density) and 15% (x-velocity). Tucker's strength is not universal: on NYX temperature, which is not low-rank, tc-wavelet beats SPERR (0.445 / 2.75 / 6.09 against 0.575 / 3.13 / 6.50 bpv at 60/80/100 dB) while Tucker falls well behind (1.18 / 4.40 / 7.95). That gap is the reason the chooser exists. [M1 §2]

The learned context model adds a further, smaller step (Figure 4). With the transform and predictor held fixed, it saves 3–7% over the hand context model; at 80 dB the overfit network saves 6.6% on Miranda density, 5.8% on x-velocity and 3.0% on Hurricane U. The pretrained network wins at low rate, where 750 bytes of shipped weights would be a large share of a small stream; the overfit one wins at higher rate. These models were trained with field-family exclusion, so no Miranda data was used for the Miranda tests and no Hurricane-U data for the Hurricane-U test. [M1 §3]

TABLE 3 Classical stack on the development set, bpv (lower is better; best in orange). [M1 §2; S]

Field	Target	SPERR	TTHRESH	SZ3	tc-wavelet	tc-tucker
Miranda density	60 dB	0.307	0.314	0.580	0.246	**0.218**
	80 dB	1.196	0.773	2.58	1.064	**0.620**
	100 dB	2.843	1.590	5.56	2.504	**1.413**
	max 1e-4	2.005	n/a	3.40	**1.694**	n/a
Miranda velocityx	60 dB	0.191	0.222	0.352	0.172	**0.144**
	80 dB	0.876	0.618	1.98	0.816	**0.493**
	100 dB	2.347	1.534	4.87	2.163	**1.299**
	max 1e-4	1.617	n/a	2.60	**1.403**	n/a
Hurricane U	60 dB	0.323	0.397	0.809	0.338	**0.284**
	80 dB	1.711	1.421	3.67	1.680	**1.167**
	max 1e-4	2.810	n/a	4.45	**2.592**	n/a

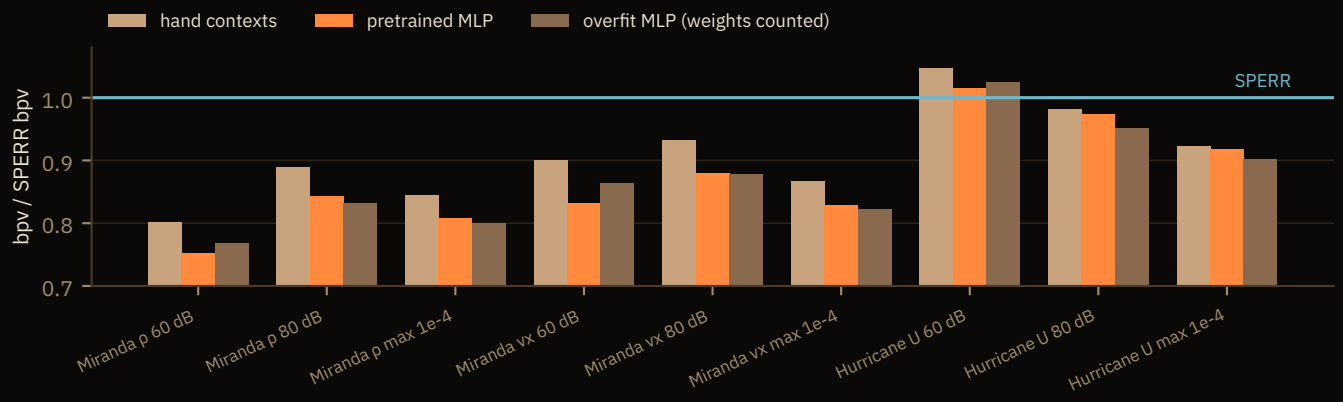


FIG 4 The learned context model on the development set, bpv relative to SPERR. Each group compares hand contexts, a pretrained MLP and an overfit MLP whose weights are counted in the rate. [M1 §3]

5.3 Held-out fields

The held-out fields are the real test, because nothing in the codec was chosen with them in view. Figure 5 shows every codec's measured rate–distortion front on each of them. On almost every panel the tc-auto curve sits to the left of the others, meaning fewer bits for the same PSNR, and the margin is as large on the new applications as on the three controls.

Summarised across fields, the frozen tc-auto uses 0.719, 0.670 and 0.693 of SPERR's bits at 60, 80 and 100 dB (Figure 6). Against a harder reference, the best external codec at each individual point, chosen with hindsight among SPERR, TTHRESH, SZ3, ZFP and QTT, it uses 0.775, 0.737 and 0.808. The frozen pretrained learned model transfers as well: on its own it reaches 0.83–0.89 of SPERR, where the hand context model reaches 0.87–0.97. [M1 §8; S]

Two further checks guard against an average hiding a weak group. Splitting by group (Table 4) shows no penalty for new applications: against the best external codec, tc-auto is at 0.741 on the eleven new-application fields at 80 dB and 0.722 on the three controls. Counting individual points (Table 5), tc-auto beats the per-point best external codec on 54 of 55 PSNR points and 34 of 42 max-abs points, with ties within 1% counted as ties. [M1 §8; S]

TABLE 4 tc-auto vs the best external codec, by group. [M1 §8; S]

Group	60 dB	80 dB	100 dB	max 1e-4
new-app	0.760	0.741	0.816	0.880
same-app	0.833	0.722	0.780	0.863

The losses are informative, and Figure 7 shows all of them. Every one but one is a max-abs point, and they fall into two kinds. On very smooth fields at the loosest bound (QMCPACK, scale-PRES, scale-T at 1e-2), the stream is only 0.01–0.06 bpv and our fixed overheads, such as centroid bytes, predictor bytes and the outlier map, become a large share of it; SZ3's leaner Lorenzo-plus-Huffman design wins

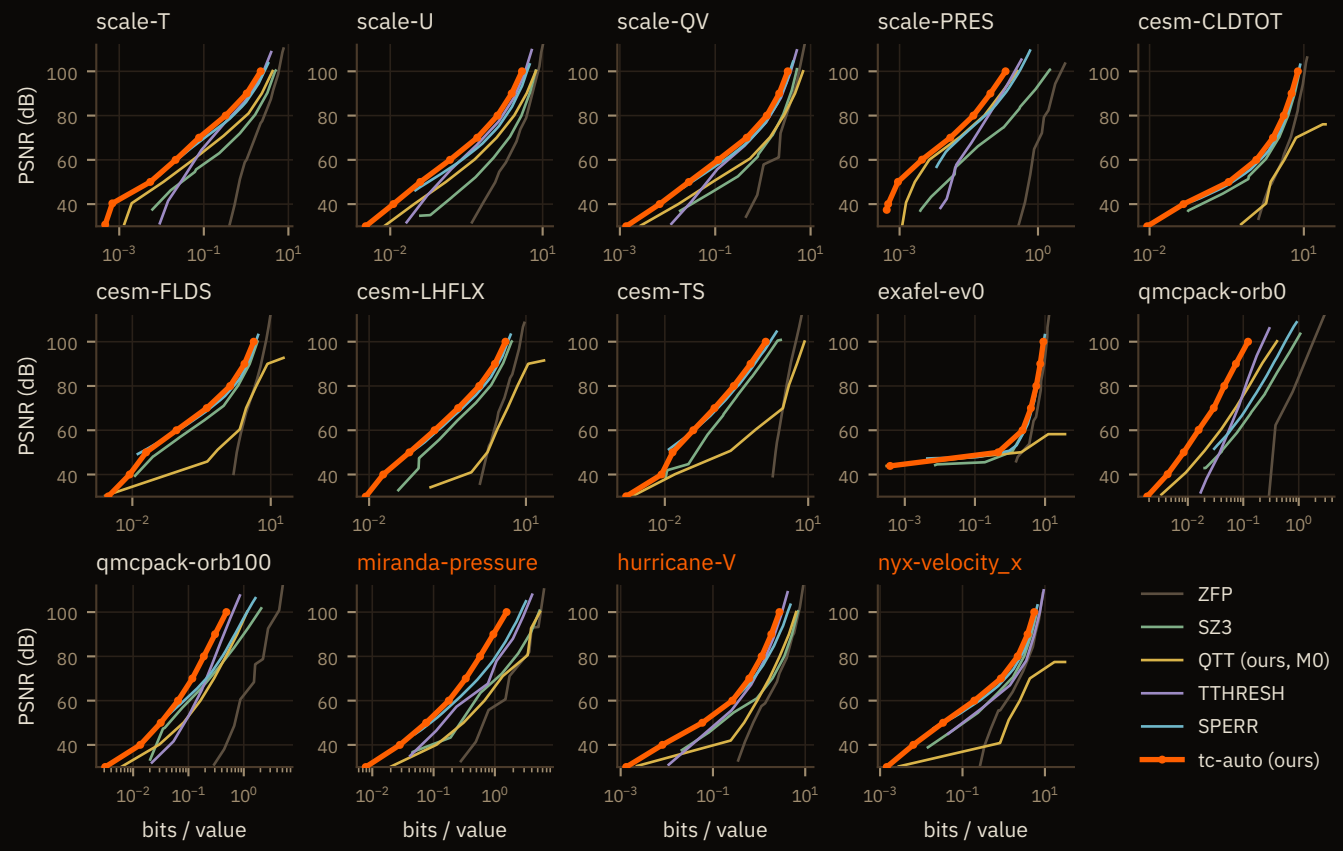


FIG 5 Rate–distortion fronts on all 14 held-out fields: measured PSNR against bpv, log x axis. Orange titles are same-application controls; the rest come from applications never seen during development. TTHRESH is absent on the 2D fields. [B heldout_*.csv]

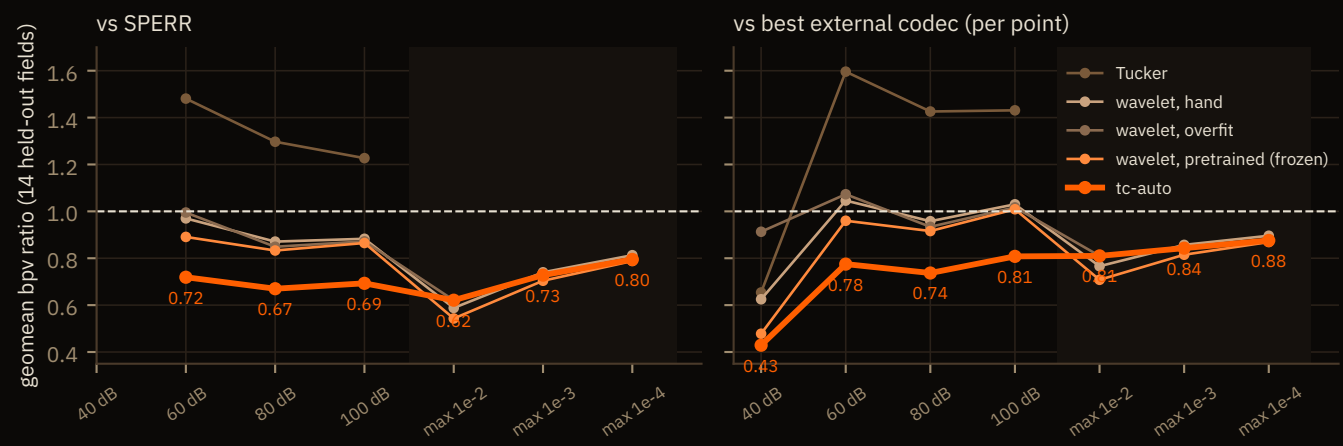


FIG 6 Geometric-mean bpv ratio over the 14 held-out fields. The shaded band holds the max-abs points, where tc-auto equals the overfit wavelet because Tucker has no max-abs mode. Tucker on its own is poor on the new applications, at 1.48× SPERR at 60 dB. [M1 §8; S]

there. On sparse, peaky data (the EXAFEL detector image, QMCPACK orbital 0) at tight bounds, the wavelet smears each sharp peak across many coefficients, the outlier stream grows, and QTT wins by 1.08–1.31×. The single PSNR loss is Hurricane-V at 60 dB, by 1.4% to SPERR. [M1 §8; S]

TABLE 5 Win–loss of tc-auto vs the best external codec; ±1% is a tie. [M1 §8; S]

Group	PSNR	Max-abs
all	**54–1**	34–8
new-app	43–0	25–8
same-app	11–1	9–0

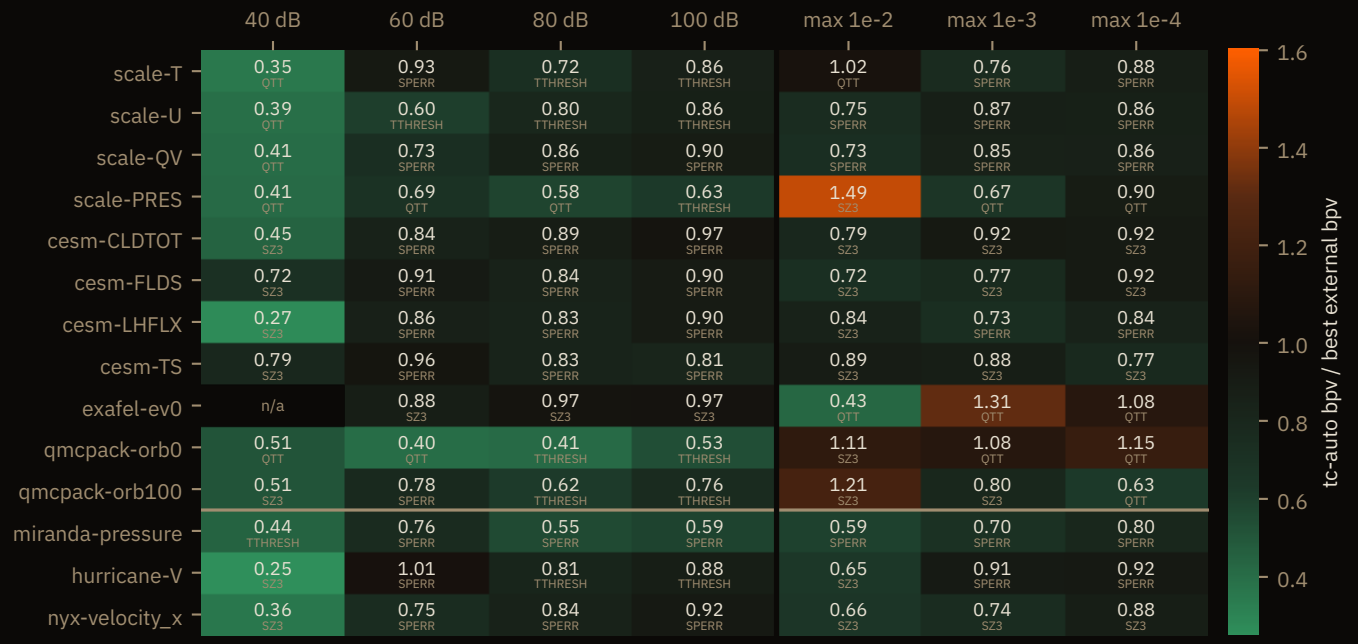


FIG 7 tc-auto bpv divided by the best external codec at each field and point, with that codec named underneath. Green means tc-auto uses fewer bits. The losses cluster in two places: loose bounds on very smooth fields and tight bounds on peaky ones. `[S, held-out per-field table]`

The chooser behaves as intended. Against an oracle that picks the better of its two candidates after the fact, tc-auto's geometric-mean ratio is 1.0004, and it is within 1% of that oracle at 95 of 97 points. Against an oracle over all four of our codecs it is at 1.068, because it never considers the pretrained model, which wins some low-rate points. It chose wavelet 90 times and Tucker 64 times. `[M1 §8; S]`

5.4 What the numbers look like

Ratios are abstract, so we also looked at reconstructions. These are new runs made for this paper, not numbers copied from the reports. Each codec was tuned on its own knob to land at or just under a common rate, decoded by its own decoder, and measured; every result is logged in `paper-draft-figures/demo_results.json`.

At about 0.03 bpv on a held-out CESM surface-temperature field (Figure 8), roughly a thousand times smaller than the input, tc-auto and SPERR are at parity, 57.5 and 57.8 dB, with SPERR using 1% more bits. SZ3 reaches 44.3 dB at the same rate, and its error map is visibly brighter across most of the field. ZFP cannot produce a stream this small for this field. We include this example because it is typical of the 2D CESM fields, where the advantage over SPERR is modest, and it would be misleading to show only the 3D cases. `[demo_results.json]`

On a 3D Miranda density slice at about 0.05 bpv (Figure 9), the gap is larger: tc-auto reaches 41.3 dB, SPERR 40.5 and SZ3 33.2. The zoom row shows the difference directly: SZ3's reconstruction loses the shape of the mixing interface that both wavelet coders keep.

Finally, Figure 10 makes the pointwise guarantee concrete. With a bound of one thousandth of the range on CESM-TS, every codec keeps every one of the 1,048,576 cells inside ±b. What differs is the price: tc-auto needs 0.201 bpv, SZ3 0.229, SPERR 0.259 and ZFP 3.55. `[demo_results.json]`

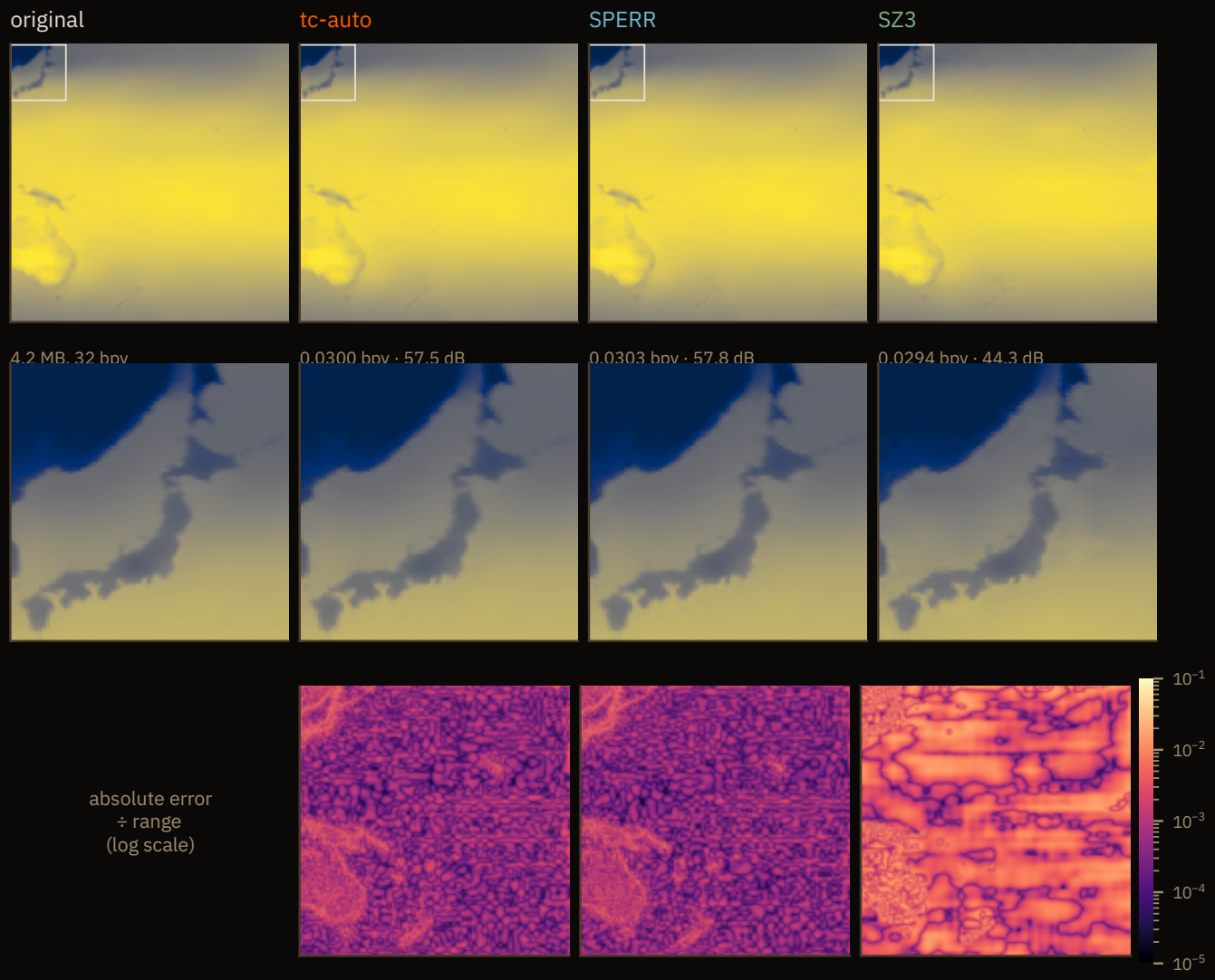
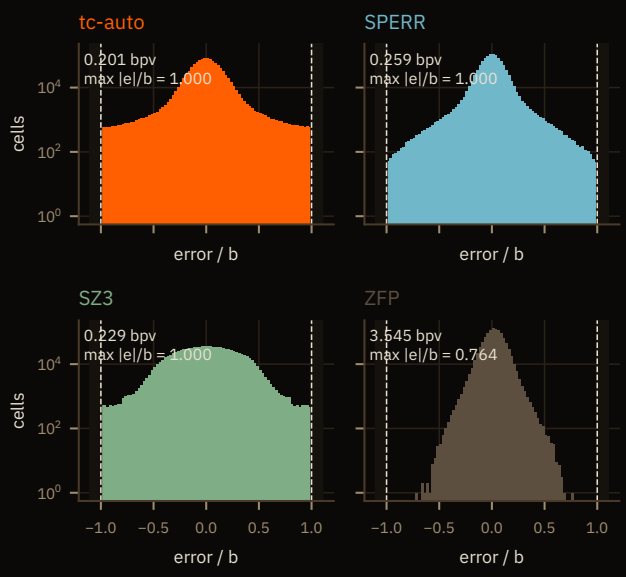


FIG 8 Equal rate on a held-out field: CESM-ATM surface temperature at about 0.03 bpv. Rows show the full field, the boxed region at full resolution, and absolute error ÷ range on a shared log scale. ZFP's smallest stream on this field is 1.31 bpv, so it is left out. `[demo_results.json]`



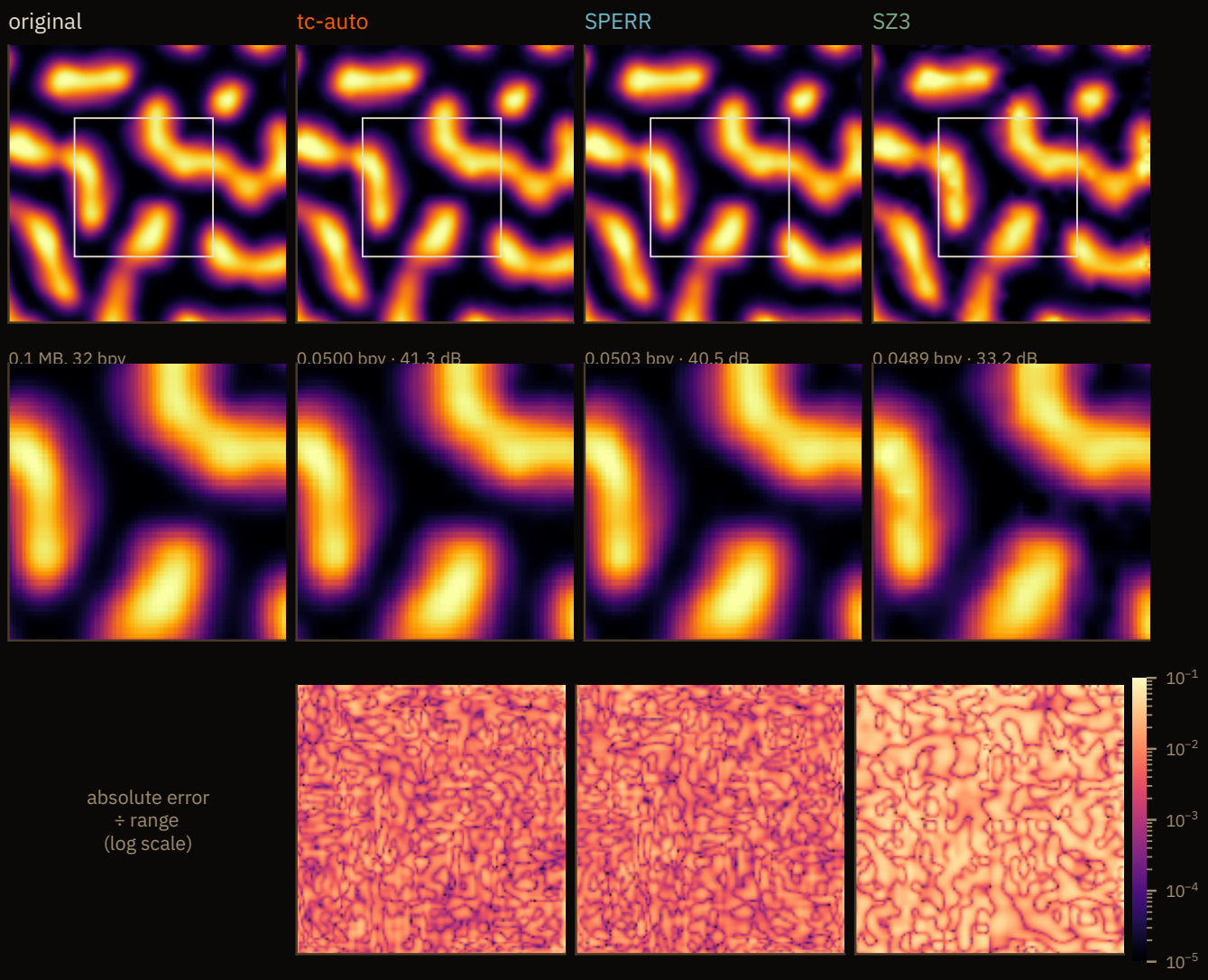


FIG 9 Equal rate on a 3D development field: Miranda density, slice z = 32 of the 64×128×128 crop, at about 0.05 bpv. ZFP's smallest stream is 0.32 bpv. [demo_results.json]

FIG 10 Pointwise bound b = 10^{-3} · range on CESM-TS: histograms of signed error ÷ b over all cells, log count. Every codec stays inside the dashed lines. [demo_results.json]

5.5 GPU decoding

The first requirement for the GPU formats was exactness, and it was met without exception. Across the TCG1 sweep (190 cases), TCG3 with one lane per block (190) and with eight (95), and TCG3 with the learned model (95), every decoded file was byte-identical between CPU and GPU, every error bound held, and every random-access sub-box matched the corresponding crop of a full decode. Both 95-case sweeps were run again after the final kernel changes, with the same result. [M1d §2, §9–11]

The second requirement, speed, was largely met. TCG1 decodes at 1.3–7.3 GB/s in kernel time on a 537 MB NYX field, and 0.43–2.1 GB/s including transfers. The learned model on TCG3 reached 1.10 GB/s in kernel time and 0.91 GB/s end to end with a 16-unit hidden layer; the larger 32×16 network reached 0.85 GB/s. For orientation, on the same field SPERR decodes at 0.095 GB/s on one thread and SZ3 at 0.59 GB/s, though those runs are not rate-matched. Figure 11 shows both views. [M1d §4, §11]

The third requirement was that parallelism should cost at most 5% in rate against the serial coder, and here we failed. Table 6 gives the numbers. Small bricks are expensive: TCG1 with 32^3 bricks needs 1.486 times the serial coder's bits at 80 dB. Moving to code-blocks over one global transform removes most of that, and TCG3 with 32^3 blocks

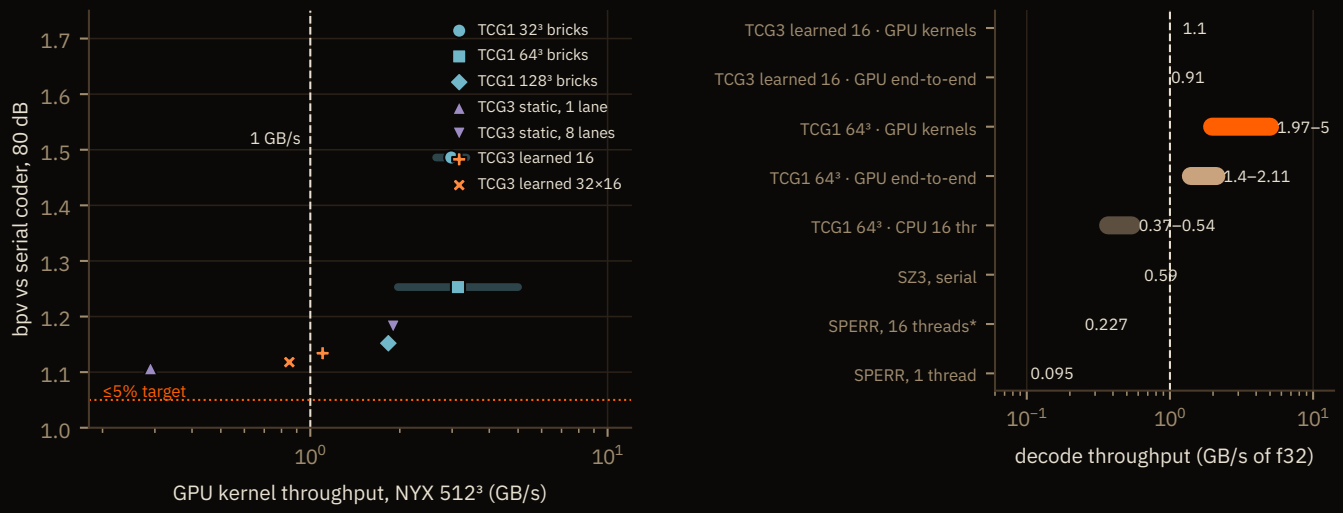


FIG 11 Left: rate cost against GPU kernel throughput on NYX 512³; bars are ranges over runs. Right: decode throughput on the same 537 MB field; SPERR and SZ3 are not rate-matched, and 16-thread SPERR includes file I/O. [M1d §3, §4, §9–11]

and one lane is at 1.107. But even with a single block per band TCG3 sits at 1.099, which tells us that what remains is the coder (static rather than adaptive probabilities, and contexts restricted by the parallel order), not the blocking. Figure 12 shows that the cost is uneven: low-rate, anisotropic atmospheric fields pay the most. [M1d §3, §9, §10]

TABLE 6 Geometric-mean bpv ratio over 19 fields at 80 dB. [M1d §3, §9, §10]

80 dB, 19 fields	vs serial	vs SPERR
TCG1 32³ bricks	1.486	1.325
TCG1 64³ bricks	1.253	1.117
TCG1 128³ / whole	1.152	1.027
TCG3 static, 1 lane	1.107	**0.987**
TCG3 static, 8 lanes	1.182	1.054
TCG3 learned 32×16	1.118	**0.997**
TCG3 learned 16	1.134	1.011
serial tc-wavelet	1	**0.892**

Seen from outside, the failure is relative. Against SPERR, TCG3 is at parity at matched PSNR (0.987 at 80 dB with static priors, 0.997 with the learned model) and better under pointwise bounds (0.816 and 0.845 at 1e-3). The GPU formats give up most of our serial advantage, but not more than that. [M1d §9–10]

Random access shows the opposite trade-off. TCG1's 32³ bricks return a 64³ box in 22–41 ms on one CPU thread. TCG3 must read 27–61% of the stream to return a box that is 0.2% of the field, because coarse levels hold much of a low-rate stream and every block's context reaches into them. [M1d §4, §9]

6

Discussion

Three lessons stand out.

The first is that most of the gain is unglamorous. The learned model is the most novel part of the system, but it contributes 3–7%. The bulk of the advantage over SPERR comes from per-band linear prediction and adaptive-Golomb contexts, techniques that a careful engineer could have added to a wavelet coder at any time in the last twenty years. Our reading is that wavelet coders have under-invested in their final stage because set partitioning made it look finished. [M1 §3]

The second is that the held-out protocol was worth its cost. It would have been easy to report development-set numbers, and they are larger: up to 25% below SPERR for the best learned configuration on Miranda density at 60 dB. The held-out numbers are what a user should expect. They held up, with no new-application penalty, which is a stronger claim than any development-set table can support. [M1 §3, §8]

The third is that determinism and parallelism pull against compression in different ways. Determinism did not show up as a visible cost: because the encoder rate-controls on the integer decoder, its effect is folded into every rate we

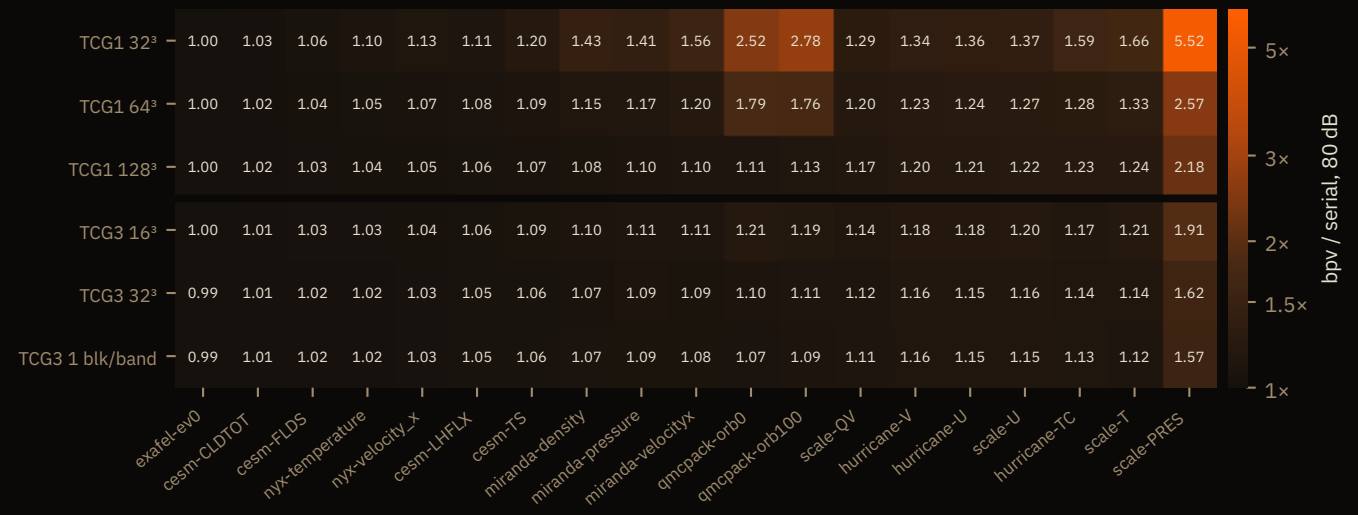


FIG 12 Per-field rate cost of each GPU layout at 80 dB relative to the serial coder, sorted by TCG1 128³. Code-blocks (lower rows) remove most of the bricking cost; scale-PRES, the SCALE fields and Hurricane pay the most. [B gpu_ratio.jsonl, gpu3_ratio.jsonl]

report, although we have not isolated it (§6). What we can measure is the cost of parallelism, which limits the neighbours a coefficient's context can see and rules out adaptive statistics. The remaining 10–15% is therefore a problem of context design under a parallel schedule, not of arithmetic, and that is where we would look next.

7

Limitations

1. **Encoding is slow.** tc-auto's median encode is 1.8 s per held-out field, against 0.04 s for SPERR and 0.007 s for SZ3, about forty times SPERR. Most of the time goes to training the overfit model and to running both candidates. The GPU formats encode on the CPU only: 16–190 s for NYX 512³. [M1 §8; M1d §4]
2. **The learned CPU decoder is slow.** It is 5–14 times slower than hand decoding (0.70 s against 0.05 s on Miranda density at 60 dB); on the held-out fields its median decode is 0.47 s against SPERR's 0.03 s. [M1 §4, §8]
3. **Loose bounds on very smooth data** favour SZ3, by 1.1–1.5× at 1e-2, because fixed overheads dominate a stream of a few hundredths of a bit per value. The summary table lists one of these points, scale-T, as a QTT win rather than SZ3; the two reports need reconciling. [M1 §8; S]
4. **Sparse, peaky data** at tight bounds favours QTT, by 1.08–1.31×. [M1 §8]
5. **The GPU ratio target was missed**: the best formats are 10.7% (static, one lane) and 11.8% (learned 32×16) above the serial coder at 80 dB. The learned model with one-lane blocks was not measured on all 19 fields. [M1d §9, §10]
6. **Throughput targets were met only in part.** 1 GB/s was reached with the 16-unit network, not the 32×16 one, and one-lane TCG3 runs at 0.17–0.64 GB/s. [M1d §9, §11]
7. **Timings are noisy**, as described in §4. [M1; M1d]
8. **The evaluation is small**: 19 cropped fields on one machine, throughput measured on one large field plus Hurricane U at 100×500×500, and only two or three fields behind each development-set pretrained model. [M1 §5; M1d §4]
9. **Exactness is shown on one GPU** (Apple M4 Max through Metal), and Tucker's reconstruction can differ in the last place across CPU instruction sets. [M1 §1]
10. **One ablation is missing**: we did not compare the fixed-point GPU transform against a floating-point one, so its cost is included in our rates but not isolated. [M1d §1]

8

Related work

The error-bounded compressors we compare against are SZ [Di16] and SZ3 [Liang22], ZFP [Lindstrom14], SPERR [Li23], which builds on SPECK [Pearlman04], and TTHRESH [BallesterRipoll20]; MGARD [Ainsworth19] is a further

multilevel approach we did not benchmark. Our data come from SDRBench [Zhao20]. The wavelet machinery is standard: CDF biorthogonal wavelets [Cohen92], lifting [Sweldens96], the code-blocks of EBCOT and JPEG 2000 [Taubman00], and set partitioning in hierarchical trees [Said96]. Our tensor codecs draw on the HOSVD [DeLathauwer00], tensor trains [Oseledets11], quantized tensor trains [Khoromskij11] and optimal singular-value thresholding [GavishDonoho14]. On the coding side we use rANS [Duda13] and range coding [Martin79], and our adaptive probability map follows PAQ-style secondary estimation [Mahoney05]. The learned model borrows its output distribution from PixelCNN++ [Salimans17], sits in the tradition of learned image compression [Balle17], and relies on the same discipline as integer-only network inference [Jacob18]. A survey of learned compressors for scientific data specifically is still to be written for this draft.

9

Reproducibility

Commands for every experiment are in M0 §7, M1 §7 and M1d §7, §9 and §11. The relevant commits on branch `claude/tang-compress-qtt` are: codecs `e422677`, `ec0346c`, `a0cfc9c`; the frozen held-out codec `4e27445` and its fix `009bec6`; TCG1 `0b706b6`, `08b708c`, `c70080c`; TCG3 `d2dfccd`, `c78eb54`; learned TCG3 `2253526`, `5972552`; GPU speedups `930adfe`, `459b737`, `75b9892`. `paper-draft-figures/make_figures.py` regenerates every plot from source data, and `make_demos.py` reruns the visual examples. No data was committed.

References

□ = DETAILS UNCERTAIN; VERIFY BEFORE CIRCULATION

[Ainsworth19] M. Ainsworth, O. Tugluk, B. Whitney, S. Klasky. Multilevel techniques for compression and reduction of scientific data — the univariate case. Comput. Vis. Sci., 2018/2019. □

[Balle17] J. Ballé, V. Laparra, E. P. Simoncelli. End-to-end optimized image compression. ICLR 2017.

[BallesterRipoll20] R. Ballester-Ripoll, P. Lindstrom, R. Pajarola. TTHRESH: Tensor compression for multidimensional visual data. IEEE TVCG 26(9), 2020.

[Cohen92] A. Cohen, I. Daubechies, J.-C. Feauveau. Biorthogonal bases of compactly supported wavelets. CPAM 45(5), 1992.

[DeLathauwer00] L. De Lathauwer, B. De Moor, J. Vandewalle. A multilinear singular value decomposition. SIMAX 21(4), 2000.

[Di16] S. Di, F. Cappello. Fast error-bounded lossy HPC data compression with SZ. IPDPS 2016.

[Duda13] J. Duda. Asymmetric numeral systems. arXiv:1311.2540, 2013.

[GavishDonoho14] M. Gavish, D. L. Donoho. The optimal hard threshold for singular values is 4/√3. IEEE Trans. Inf. Theory 60(8), 2014.

[Jacob18] B. Jacob et al. Quantization and training of neural networks for efficient integer-arithmetic-only inference. CVPR 2018.

[Khoromskij11] B. N. Khoromskij. O(d log N)-quantics approximation of N-d tensors in high-dimensional numerical modeling. Constr. Approx. 34, 2011. □

[Li23] S. Li, P. Lindstrom, J. Clyne. Lossy scientific data compression with SPERR. IPDPS 2023.

[Liang22] X. Liang, K. Zhao, S. Di, et al. SZ3: A modular framework for composing prediction-based error-bounded lossy compressors. IEEE Trans. Big Data, 2022/2023. □

[Lindstrom14] P. Lindstrom. Fixed-rate compressed floating-point arrays. IEEE TVCG 20(12), 2014.

[Mahoney05] M. Mahoney. Adaptive weighing of context models for lossless data compression. Florida Tech. TR CS-2005-16, 2005. □

[Martin79] G. N. N. Martin. Range encoding: an algorithm for removing redundancy from a digitised message. 1979.

[Oseledets11] I. V. Oseledets. Tensor-train decomposition. SIAM J. Sci. Comput. 33(5), 2011.

[Pearlman04] W. A. Pearlman, A. Islam, N. Nagaraj, A. Said. Efficient, low-complexity image coding with a set-partitioning embedded block coder. IEEE TCSVT 14(11), 2004.

[Said96] A. Said, W. A. Pearlman. A new, fast, and efficient image codec based on set partitioning in hierarchical trees. IEEE TCSVT 6(3), 1996.

[Salimans17] T. Salimans, A. Karpathy, X. Chen, D. P. Kingma. PixelCNN++. ICLR 2017.

[Sweldens96] W. Sweldens. The lifting scheme: a custom-design construction of biorthogonal wavelets. ACHA 3(2), 1996.

[Taubman00] D. Taubman. High performance scalable image compression with EBCOT. IEEE TIP 9(7), 2000.

[Zhao20] K. Zhao, S. Di, X. Liang, et al. SDRBench: Scientific data reduction benchmark for lossy compressors. IEEE BigData 2020 (workshop). □